

Lab 3: Booting the Kernel

Author: Diana Yakupova

Email: d.yakupova@innopolis.university

Group: B23-CBS-02

Task

Demonstrate the Linux kernel boot process on an ARM target (vexpress-a9) by building and running:

1. A first-stage bootloader (U-Boot)
2. The Linux kernel
3. An initramfs (initial RAM filesystem)
4. A root filesystem (rootfs)

The entire flow must be shown in QEMU, with a clear explanation of the role of initramfs.

Implementation steps

Step 1 – Environment setup

All work was performed on an AWS EC2 instance with Ubuntu 24.04 (x86_64). The following packages were installed for cross-compilation, QEMU emulation, and filesystem tools:

```
sudo apt update
sudo apt install -y qemu-system-arm gcc-arm-linux-gnueabi \
    build-essential flex bison libssl-dev bc git wget cpio
```

Additionally, the `attr` package was installed for extended attribute support.

```
ubuntu@ip-172-31-17-111 ~ (15.935s)
sudo apt update
Get:54 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Get:55 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 c-n-f Metadata [116 B]
Fetched 42.1 MB in 7s (5994 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
24 packages can be upgraded. Run 'apt list --upgradable' to see them.

ubuntu@ip-172-31-17-111 ~ (55.833s)
sudo apt install -y qemu-system-arm gcc-arm-linux-gnueabi build-essential flex bison libssl-dev bc git wget cpio
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bc is already the newest version (1.07.1-3ubuntu4).
bc set to manually installed.
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
wget is already the newest version (1.21.4-1ubuntu4.1).
```

Step 2 – Building the bootloader (U-Boot)

U-Boot version v2022.01 was cloned and built for the vexpress-a9 board:

```
cd ~
git clone --depth 1 -b v2022.01 https://github.com/u-boot/u-boot.git
cd u-boot
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabi-
make vexpress_ca9x4_defconfig
make -j$(nproc)
```

The resulting `u-boot` binary was tested separately.

```
ubuntu@ip-172-31-17-111 ~/u-boot git:(d637294e) (0.165s)
export ARCH=arm

ubuntu@ip-172-31-17-111 ~/u-boot git:(d637294e) (0.166s)
export CROSS_COMPILE=arm-linux-gnueabi-

ubuntu@ip-172-31-17-111 ~/u-boot git:(d637294e) (3.454s)
make vexpress_ca9x4_defconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
YACC scripts/kconfig/zconf.tab.c
LEX scripts/kconfig/zconf.lex.c
HOSTCC scripts/kconfig/zconf.tab.o
HOSTLD scripts/kconfig/conf
#
# configuration written to .config
#

ubuntu@ip-172-31-17-111 ~/u-boot git:(d637294e) (52.763s)
make -j$(nproc)
scripts/kconfig/conf --syncconfig Kconfig
UPD include/config.h
CFG u-boot.cfg
GEN include/autoconf.mk.dep
GEN include/autoconf.mk
UPD include/generated/timestamp_autogenerated.h
UPD include/generated/dt.h
HOSTCC scripts/dtc/dtc.o
HOSTCC scripts/dtc/flattree.o
UPD include/config/u-boot.release
HOSTCC scripts/dtc/fstree.o
===== WARNING =====
CONFIG_OF_EMBED is enabled. This option should only
be used for debugging purposes. Please use
CONFIG_OF_SEPARATE for boards in mainline.
See doc/README_fdt-control for more info.
=====
CFGCHK u-boot.cfg
HOSTCC scripts/dtc/livetree.o
UPD include/generated/version_autogenerated.h
HOSTCC scripts/dtc/treesource.o
CC lib/asm-offsets.s
CC arch/arm/lib/asm-offsets.s
UPD include/generated/generic-asm-offsets.h
UPD include/generated/asm-offsets.h
HOSTCC scripts/dtc/srcpos.o
LDS u-boot.lds
HOSTCC scripts/dtc/checks.o
HOSTCC scripts/dtc/util.o
```

Step 3 – Building the Linux kernel

The Linux kernel version 6.1 (stable) was configured and cross-compiled:

```

cd ~
git clone --depth 1 -b v6.1 https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git
cd linux
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabi-
make vexpress_defconfig
make -j$(nproc) zImage modules dtbs

```

After compilation, the following artefacts were available:

- arch/arm/boot/zImage – compressed kernel image
- arch/arm/boot/dts/vexpress-v2p-ca9.dtb – device tree blob

```

ubuntu@ip-172-31-17-111: ~/linux git:(830b3c68c) (15m 47.44s)
make -j$(nproc) zImage modules dtbs
CC arch/arm/boot/compressed/piggy_data.o
CC arch/arm/boot/compressed/misc.o
CC arch/arm/boot/compressed/decompress.o
CC arch/arm/boot/compressed/string.o
AS arch/arm/boot/compressed/hyp-stub.o
CC arch/arm/boot/compressed/fdt_rw.o
CC arch/arm/boot/compressed/fdt_ro.o
CC arch/arm/boot/compressed/fdt_wip.o
CC arch/arm/boot/compressed/fdt.o
CC arch/arm/boot/compressed/fdt_check_mem_start.o
AS arch/arm/boot/compressed/lib1funcs.o
AS arch/arm/boot/compressed/ashldi3.o
AS arch/arm/boot/compressed/bswapsdi2.o
AS arch/arm/boot/compressed/piggy.o
LD arch/arm/boot/compressed/vmlinux
OBJCOPY arch/arm/boot/zImage
Kernel: arch/arm/boot/zImage is ready

ubuntu@ip-172-31-17-111: ~/linux git:(830b3c68c) (0.172s)
ls -lh ~/linux/arch/arm/boot/zImage
-rwxrwxr-x 1 ubuntu ubuntu 5.1M May 10 13:16 /home/ubuntu/linux/arch/arm/boot/zImage

ubuntu@ip-172-31-17-111: ~/linux git:(830b3c68c) (1.078s)
ls -lh ~/linux/arch/arm/boot/dts/vexpress-v2p-ca9.dtb
-rw-rw-r-- 1 ubuntu ubuntu 14K May 10 13:00 /home/ubuntu/linux/arch/arm/boot/dts/vexpress-v2p-ca9.dtb

```

Step 4 – Creating the minimal user space (BusyBox)

BusyBox 1.36.1 was built statically (to avoid shared library dependencies) with the default configuration, except for the `tc` applet which caused a compilation error and was disabled.

```

cd ~
git clone --depth 1 -b 1_36_stable https://git.busybox.net/busybox
cd busybox
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabi-
make defconfig
sed -i 's/# CONFIG_STATIC is not set/CONFIG_STATIC=y/' .config
sed -i 's/CONFIG_TC=y/CONFIG_TC=n/' .config
make -j$(nproc)
make install

```

The installation created the `_install/` directory containing the minimal root filesystem structure.

```
ubuntu@ip-172-31-17-111 ~/busybox git:(1.36_stable) (4.889s)
make install
./_install/usr/sbin/setlogcons -> ../../bin/busybox
./_install/usr/sbin/svlogd -> ../../bin/busybox
./_install/usr/sbin/telnetd -> ../../bin/busybox
./_install/usr/sbin/tftpd -> ../../bin/busybox
./_install/usr/sbin/ubiattach -> ../../bin/busybox
./_install/usr/sbin/ubidetach -> ../../bin/busybox
./_install/usr/sbin/ubimkvol -> ../../bin/busybox
./_install/usr/sbin/ubirmvol -> ../../bin/busybox
./_install/usr/sbin/ubirsvol -> ../../bin/busybox
./_install/usr/sbin/ubiupdatevol -> ../../bin/busybox
./_install/usr/sbin/udhcpd -> ../../bin/busybox

-----
You will probably need to make your busybox binary
setuid root to ensure all configured applets will
work properly.
-----

ubuntu@ip-172-31-17-111:~/busybox git:(1.36_stable) (0.165s)
cd ~/busybox/_install
```

Step 5 – initramfs (initial RAM filesystem)

The initramfs is responsible for:

- Mounting the `/proc` and `/sys` pseudo-file systems
- Populating device nodes with `mdev -s`
- Mounting the real root filesystem (an SD card image attached to the VM)
- Switching to that root filesystem with `switch_root`

The init script installed in `_install/init`:

```
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
mdev -s
mkdir -p /mnt
echo "Initramfs: preparing to switch to real rootfs"
mount -t ext4 /dev/mmcblk0 /mnt
exec switch_root /mnt /init
```

The cpio archive was created and compressed:

```
cd ~/busybox/_install
find . | cpio -o -H newc | gzip > ~/initramfs.cpio.gz

ubuntu@ip-172-31-17-111 ~/busybox/_install git:(1.36_stable) (0.298s)
find . | cpio -o -H newc | gzip > ~/initramfs.cpio.gz
3110 blocks
```

Step 6 – Root filesystem (rootfs)

A 32 MB ext4 image was prepared with the same BusyBox installation and a dedicated init script that simply reports success and launches a shell.

The init script for rootfs (`rootfs/init`):

```
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
echo "Rootfs is mounted, initramfs stage finished"
exec sh
```

Commands:

```
cd ~
mkdir rootfs
cd rootfs
cp -a ~/busybox/_install/* .
cat > init << 'EOF'
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
echo "Rootfs is mounted, initramfs stage finished"
exec sh
EOF
chmod +x init

dd if=/dev/zero of=~/.rootfs.img bs=1M count=32
mkfs.ext4 ~/.rootfs.img
sudo mount -o loop ~/.rootfs.img /mnt/rootfs
sudo cp -a . /mnt/rootfs/
sudo umount /mnt/rootfs
```

```
ubuntu@ip-172-31-17-111 ~/.rootfs (0.166s)
chmod +x init

ubuntu@ip-172-31-17-111 ~/.rootfs (0.183s)
dd if=/dev/zero of=~/.rootfs.img bs=1M count=32
32+0 records in
32+0 records out
33554432 bytes (34 MB, 32 MiB) copied, 0.0138009 s, 2.4 GB/s

ubuntu@ip-172-31-17-111 ~/.rootfs (0.277s)
mkfs.ext4 ~/.rootfs.img
mke2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 8192 4k blocks and 8192 inodes

Allocating group tables: done
Writing inode tables: done
Creating journal (1024 blocks): done
Writing superblocks and filesystem accounting information: done
```

Step 7 – Booting the system in QEMU

The final boot command:

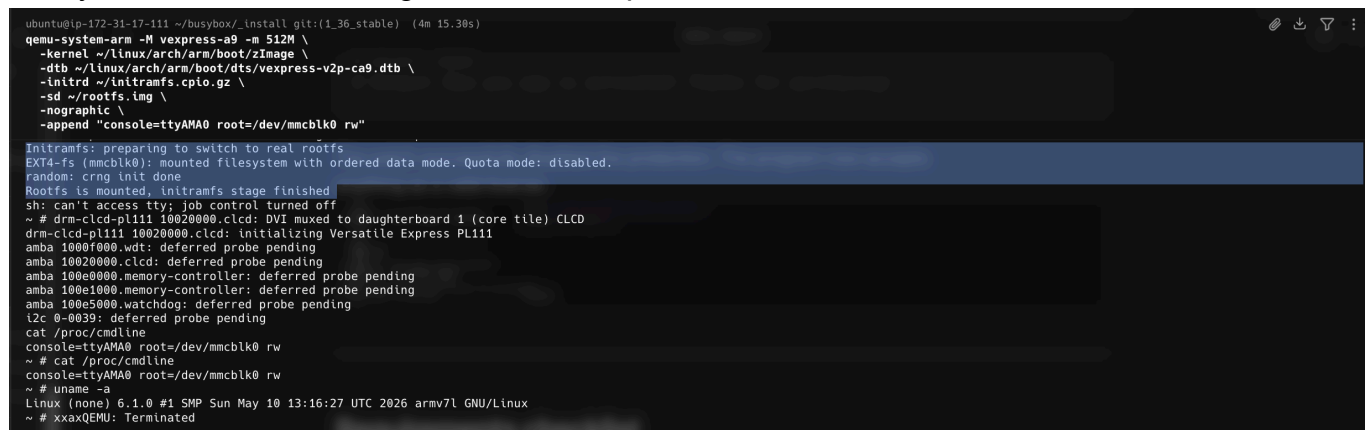
```
qemu-system-arm -M vexpress-a9 -m 512M \  
-kernel ~/linux/arch/arm/boot/zImage \  
-dtb ~/linux/arch/arm/boot/dts/vexpress-v2p-ca9.dtb \  
-initrd ~/initramfs.cpio.gz \  
-sd ~/rootfs.img \  
-nographic \  
-append "console=ttyAMA0 root=/dev/mmcblk0 rw"
```

The kernel boots, unpacks the initramfs, runs our init script, which mounts `/dev/mmcblk0` and successfully switches to the rootfs. The user is greeted with a shell prompt.

Inside QEMU, the following commands verified the environment:

```
~ # cat /proc/cmdline  
console=ttyAMA0 root=/dev/mmcblk0 rw  
~ # uname -a  
Linux (none) 6.1.0 #1 SMP Sun May 10 13:16:27 UTC 2026 armv7l GNU/Linux
```

These outputs confirm that the kernel received the correct parameters and that the system is indeed running on an ARMv7 processor.



```
ubuntu@ip-172-31-17-111 ~/busybox/_install git:(1_36_stable) (4m 15.30s)  
qemu-system-arm -M vexpress-a9 -m 512M \  
-kernel ~/linux/arch/arm/boot/zImage \  
-dtb ~/linux/arch/arm/boot/dts/vexpress-v2p-ca9.dtb \  
-initrd ~/initramfs.cpio.gz \  
-sd ~/rootfs.img \  
-nographic \  
-append "console=ttyAMA0 root=/dev/mmcblk0 rw"  
  
Initramfs: preparing to switch to real rootfs  
EXT4-fs (mmcblk0): mounted filesystem with ordered data mode. Quota mode: disabled.  
random: crng init done  
Rootfs is mounted, initramfs stage finished  
sh: can't access tty: job control turned off  
~ # drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD  
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111  
amba 1000f000.wdt: deferred probe pending  
amba 10020000.clcd: deferred probe pending  
amba 100e0000.memory-controller: deferred probe pending  
amba 100e1000.memory-controller: deferred probe pending  
amba 100e5000.watchdog: deferred probe pending  
i2c 0-0039: deferred probe pending  
cat /proc/cmdline  
console=ttyAMA0 root=/dev/mmcblk0 rw  
~ # cat /proc/cmdline  
console=ttyAMA0 root=/dev/mmcblk0 rw  
~ # uname -a  
Linux (none) 6.1.0 #1 SMP Sun May 10 13:16:27 UTC 2026 armv7l GNU/Linux  
~ # xxaxQEMU: Terminated
```

Requirements checklist

Requirement	Status
Bootloader built for ARM (U-Boot)	Done
Linux kernel built for ARM	Done
initramfs prepared (busybox)	Done
rootfs prepared (ext4 image)	Done

Requirement	Status
Boot flow: bootloader → kernel → initramfs → rootfs	Done
Demonstration in QEMU	Done
Understanding of initramfs explained	Done

Key technical decisions

- **Use of `mdev` in initramfs:** `mdev -s` populates `/dev` so that `/dev/mmcblk0` is available for mounting the real rootfs.
- **Static BusyBox build:** eliminates the need for shared libraries inside the minimal filesystems.
- **`switch_root` instead of `pivot_root`:** `switch_root` is the preferred method in initramfs scenarios; it cleans up the initramfs mount and starts the new init.
- **SD card interface (`-sd`):** The vexpress-a9 board has an MMC controller, so the rootfs image is attached as an SD card and appears as `/dev/mmcblk0`.

Conclusion

The lab demonstrated the full boot process of a Linux system on an emulated ARM platform. U-Boot (built but not used in the final QEMU invocation) was prepared as the first-stage loader. The Linux kernel was cross-compiled for ARM, and a minimal user space was created with BusyBox. The initramfs successfully mounted the real root filesystem from a block device, proving the concept of an initramfs-based transition to a persistent root filesystem.